

Smart MCQ Solver

BSDA2001P: Introduction to DL and GenAI Project

Om Manish Makadia (23f3001593)

1. Abstract

This project addresses the Smart MCQ Solver Challenge on Kaggle: given a question and five answer options (A-E), predict the top-3 most likely correct answers, ranked by confidence, evaluated using Mean Average Precision at 3 (MAP@3). The dataset consists of prompt-option pairs with no shared context passage, making the task a pure test of a model's reasoning and ranking ability rather than reading comprehension over a fixed document.

Three architecturally distinct models were built and evaluated end-to-end: (1) a Bi-LSTM with attention over FastText embeddings, trained entirely from scratch; (2) DeBERTa-v3-base fine-tuned with LoRA (PEFT) for multiple-choice classification; and (3) Qwen2.5-3B-Instruct, fine-tuned with QLoRA and augmented with a custom retrieval-augmented generation (RAG) pipeline built over a live Wikipedia corpus. All experiments were tracked using Weights & Biases with versioned model artifacts.

Individually, the three models achieved submission MAP@3 scores of 0.7344, 0.7535, and 0.7510 respectively. A test-score-informed weighted ensemble of all three achieved a public leaderboard score of 0.7577, ranking 320 out of 1531 (top 21%) on the competition leaderboard. The final pipeline was containerized with Docker and deployed as a live, interactive demo on Google Cloud Run. Key learnings spanned parameter-efficient fine-tuning (LoRA/QLoRA), retrieval-augmented generation, experiment tracking discipline, and production ML deployment under real memory and infrastructure constraints.

2. Introduction

2.1 Problem Statement

The Smart MCQ Solver Challenge tasks participants with building systems that predict the top-3 most likely correct answers for challenging multiple-choice questions, each with five options (A-E). Submissions are evaluated using Mean Average Precision at 3 (MAP@3), which rewards placing the correct answer earlier in the predicted ranking: a correct answer at rank 1 scores 1.0, at rank 2 scores 0.5, at rank 3 scores 0.333, and if absent from the top-3, scores 0. The dataset provides only a prompt and five options per question, with no shared reading-comprehension passage, meaning each model must draw on its own reasoning or retrieved external knowledge to judge option plausibility.

2.2 Project Objective

Beyond maximizing the competition metric, this project's objective was to design and compare three architecturally distinct modeling approaches: a model built entirely from scratch, a fine-tuned pretrained transformer, and a third model of choice. This project additionally aimed to gain hands-on experience with parameter-efficient fine-tuning (LoRA, QLoRA), retrieval-augmented generation, systematic experiment tracking via Weights & Biases, ensembling strategies, and end-to-end deployment of a multi-model inference pipeline.

2.3 Report Structure

Section 3 describes the dataset and preprocessing pipeline, including exploratory data analysis. Section 4 covers the tokenization strategies used across the three models. Section 5 details the architecture, design rationale, and fine-tuning strategy for each of the three models. Section 6 presents training performance, a comparative analysis across models, and final Kaggle leaderboard results. Section 7 reflects on key learnings, challenges, and future work. Section 8 lists references.

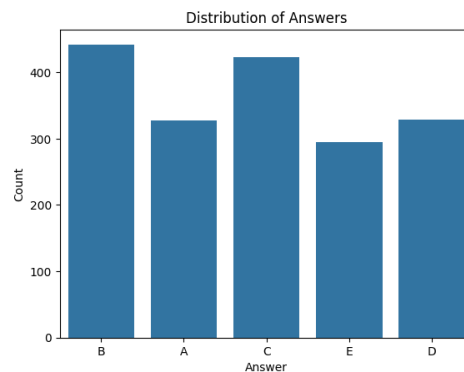
3. Dataset & Preprocessing

3.1 Dataset Description

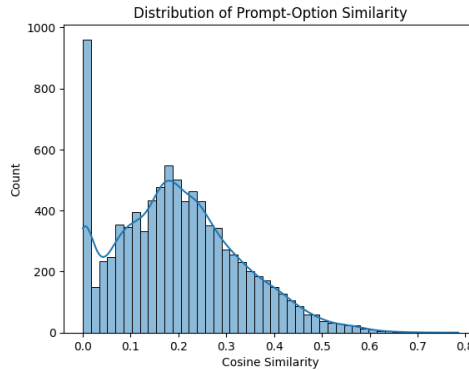
The dataset consists of multiple-choice questions, each with a unique id, a prompt (the question text), five options (columns A-E), and an answer column giving the single correct label. The dataset contains no shared context passage per question - each row is a self-contained prompt-option set, requiring models to rely on either learned world knowledge (pretrained models), pattern-matching over surface features (the from-scratch model), or explicitly retrieved external context (the RAG-augmented model).

3.2 Exploratory Data Analysis (EDA)

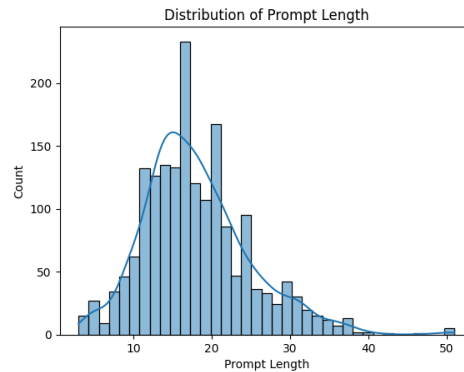
- Duplicate and leakage checks: 183 fully duplicate rows (identical prompt, options, and answer) were identified and removed. Separately, 59 duplicate prompts were found where the same question text appeared more than once - these were investigated as a potential data-quality/leakage risk before finalizing the train-validation split.
- Class balance: the answer distribution across the five labels was imbalanced, with B and C options over-represented (442 and 423 occurrences respectively) relative to E (295 occurrences), out of roughly 1,800 total questions.



- Prompt-option similarity: TF-IDF vectorization followed by cosine similarity was computed between each prompt and its five options, and between options themselves, to understand how lexically close correct vs. incorrect options tend to be - informing the decision that a purely lexical-similarity baseline would be insufficient, motivating semantic (embedding-based) and retrieval-based approaches.



- Prompt length distribution was examined to set appropriate maximum sequence lengths per model (e.g., `max_len_question=32` for Model 1; `max_length=128` for Model 2; `max_length=824` for Model 3, which needed to accommodate retrieved RAG context in addition to the prompt and options).



3.3 Data Preprocessing

- Text normalization: lowercasing and punctuation-aware regex cleaning (spacing out punctuation, collapsing whitespace) applied consistently across all three models' input pipelines.
- Deduplication: exact duplicate rows dropped prior to splitting.
- Train/validation split: a stratified split (`test_size=0.2`, `random_state=42`) was used consistently across all three models - grouped and stratified by answer label - to ensure a fair, comparable validation set for all three architectures and for later ensemble weight selection.

3.4 Data Augmentation

No explicit data augmentation (e.g., paraphrasing, back-translation) was applied. Instead, Model 3's RAG component effectively augments each question with additional external context retrieved from Wikipedia at inference time, which served a similar purpose to augmentation - enriching the information available to the model without altering the original labelled examples.

4. Tokenization Strategy

4.1 Primary Tokenizer Selection

The best-performing individual model (Model 2, DeBERTa-v3-base) uses the tokenizer associated with microsoft/deberta-v3-base - a SentencePiece-based subword tokenizer trained jointly with the model's ELECTRA-style pretraining objective.

Justification: Subword tokenization strikes a balance between the vocabulary compactness of word-level tokenization and the robustness of character-level tokenization - it handles out-of-vocabulary and rare terms gracefully (important given the varied, exam-style vocabulary in this dataset) while keeping sequence lengths manageable. Because DeBERTa-v3 was pretrained end-to-end with this tokenizer, using it preserves alignment with the model's learned subword representations, which is essential for effective fine-tuning.

4.2 Implementation Details

Tokenization for Model 2 was implemented via the Hugging Face tokenizers library, batch-processing each (prompt, option) pair as a sentence pair per the AutoModelForMultipleChoice convention - the prompt repeated once per option, paired with each of the five options respectively, then reshaped into a (batch, 5, seq_len) tensor. Padding was applied dynamically per batch (via a data collator) and truncation was capped at max_length=128 tokens, chosen based on the prompt/option length distribution from the EDA stage.

4.3 Experimental Comparison

Two other tokenization strategies were used elsewhere in the project for comparison and architectural necessity, rather than as competing candidates for the same model:

- Model 1 (from scratch) used simple whitespace-level tokenization paired with a task-specific, vocabulary-filtered FastText embedding table - appropriate for a lightweight model with no pretrained backbone, where a full subword tokenizer would add complexity without a pretrained embedding space to exploit.
- Model 3 (Qwen2.5-3B) used its native byte-level BPE tokenizer, required for compatibility with its pretrained and instruction-tuned weights, and additionally needed to tokenize longer sequences that include RAG-retrieved context.

5. Modeling & Experimentation

Three architecturally distinct models were developed, in the following order: a from-scratch neural model (Model 1), a fine-tuned pretrained transformer encoder (Model 2), and a fine-tuned generative LLM augmented with retrieval (Model 3, the project's "model of choice").

5.1 Model 1: Bi-LSTM with Attention over FastText Embeddings (From Scratch)

Architecture

Model 1 is a from-scratch neural architecture with no pretrained transformer backbone. A shared encoder - FastText embedding lookup followed by a bidirectional LSTM (hidden_dim=64, so 128-d output) and a learned attention-pooling layer - is applied independently to the question text and to each of the five option texts. The resulting question-context and each option-context vector are concatenated

(256-d) and passed through a small fully-connected classifier that outputs a single logit per option; softmax over the five per-option logits yields the final ranked probabilities.

Model 1: BiLSTM + Attention over FastText Embeddings

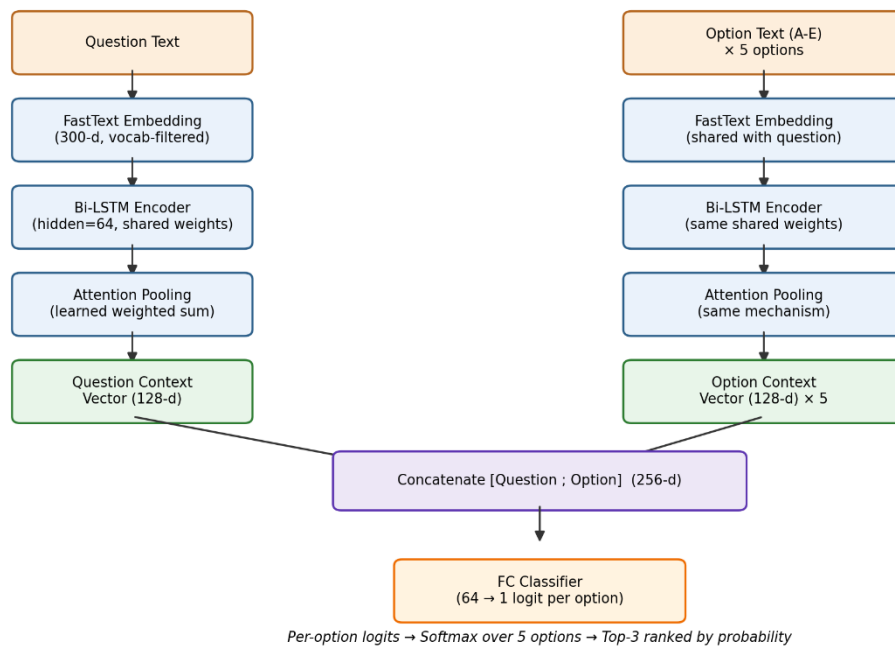


Figure 1. Architecture diagram of Model 1 - Bi-LSTM + Attention over FastText embeddings, with a shared encoder applied to the question and each option.

Salient Points

- FastText embeddings (300-d) were pre-filtered to only the vocabulary present in the training/validation data, keeping the embedding table small and task-specific rather than loading the full multi-gigabyte FastText vocabulary.
- Embeddings were fine-tuned during training (not frozen), allowing the model to adapt general-purpose word vectors to this specific MCQ task.
- The attention-pooling layer (rather than simple last-hidden-state or mean pooling) lets the model learn to weight the most informative tokens in a variable-length question or option when forming its fixed-length context vector.
- Sharing encoder weights between the question branch and the option branch reduces parameter count and encourages the model to represent questions and options in a common semantic space, which is what the final concatenation-based classifier relies on.

Fine-Tuning Strategy (Training from Scratch)

- Optimizer: Adam, learning rate = $1e-3$.
- Loss: Cross-entropy over the 5 option logits (standard 5-way classification framing).
- Batch size: 16; Epochs: up to 15, with early stopping (patience = 3 epochs) monitored on validation MAP@3.
- No layers were frozen - as a from-scratch model, all parameters (embeddings, LSTM, attention, classifier) were trained jointly.
- Quantization: not applicable - the model is small enough to train and run in full precision throughout.

5.2 Model 2: DeBERTa-v3-base Fine-Tuned with LoRA (Pretrained Model)

Architecture

Model 2 uses microsoft/deberta-v3-base (He, Gao & Chen, 2021 - “DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing”), built on the original DeBERTa architecture (He et al., 2020) which introduces disentangled attention (separately representing content and relative-position information) and an enhanced mask decoder. DeBERTa-v3 additionally replaces DeBERTa's original masked-language-modeling pretraining with ELECTRA-style replaced-token detection, improving sample efficiency and downstream task performance. The model is used via Hugging Face's AutoModelForMultipleChoice head: each (prompt, option) pair is encoded independently, the pooled [CLS]-style representation is projected to a single per-choice logit, and softmax is applied across the five choices.

Model 2: DeBERTa-v3-base + LoRA (Multiple-Choice Head)

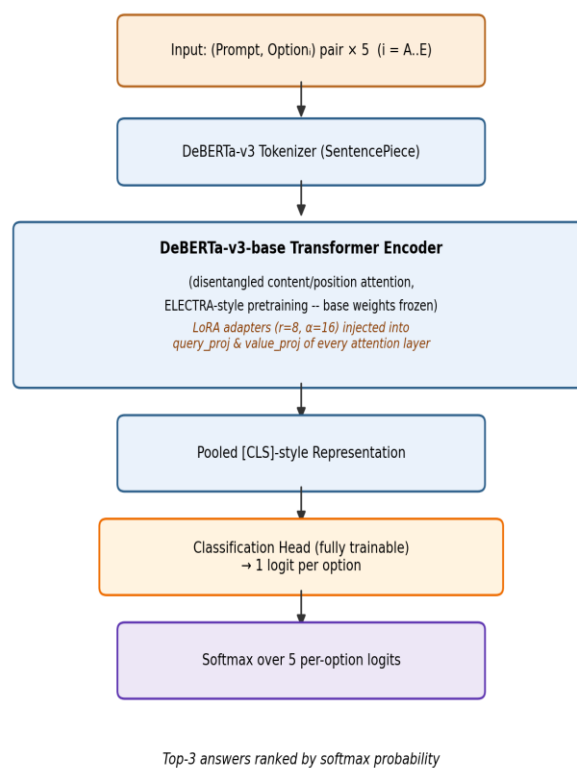


Figure 2. Architecture diagram of Model 2 - DeBERTa-v3-base with LoRA adapters in the attention layers, feeding a trainable multiple-choice classification head.

Salient Points

- Disentangled attention allows the model to reason separately about what a token is and where it is positioned relative to others - useful for MCQ reasoning, where subtle wording differences between similar options can change correctness.
- ELECTRA-style pretraining gives DeBERTa-v3 stronger sample efficiency than comparable masked-language-model-pretrained encoders, which is valuable given this competition's relatively modest dataset size (~1,800 questions).

Fine-Tuning Strategy

- Parameter-efficient fine-tuning via LoRA (PEFT): rank $r=8$, $\alpha=16$, dropout=0.1, applied to the query_proj and value_proj attention projection matrices; the classifier and pooler heads were kept fully trainable (modules_to_save) since they are task-specific and randomly initialized.
- Optimizer: AdamW (Hugging Face Trainer default) with a linear warmup schedule (warmup_ratio=0.06) and weight_decay=0.01.
- Learning rate: $2e-4$; effective batch size 16 (per_device_train_batch_size=4 with gradient_accumulation_steps=4); eval batch size 8.
- Epochs: up to 15, with early stopping (patience = 3) monitored on validation MAP@3, using load_best_model_at_end to restore the best checkpoint.
- Only LoRA adapter weights and the task-specific heads were updated; the vast majority of DeBERTa-v3-base's pretrained parameters remained frozen - this is the essence of parameter-efficient fine-tuning, keeping training fast and memory-light while retaining the pretrained model's general language understanding.
- Quantization: not applied for this model. LoRA was used without 4-bit base-model quantization (i.e. not QLoRA) since DeBERTa-v3-base is small enough to fine-tune comfortably in full precision.

5.3 Model 3: QLoRA-Fine-Tuned Qwen2.5-3B-Instruct with Retrieval-Augmented Generation (Model of Choice)

Architecture

The base model is Qwen/Qwen2.5-3B-Instruct (Qwen Team, Alibaba Group, 2024), a decoder-only causal transformer with approximately 3.09 billion parameters, using rotary positional embeddings (RoPE) and grouped-query attention for efficient inference. On top of this, a retrieval-augmented generation (RAG) pipeline was built:

- A dedicated corpus-construction pipeline (built in a separate notebook, rag_index_builder.ipynb) queries the live Wikipedia API for each cleaned question prompt, retrieves and chunks relevant article text (150-word chunks with 30-word overlap), and embeds each chunk using the BAAI/bge-large-en-v1.5 bi-encoder.
- Chunks are indexed in a FAISS index for fast similarity search; at inference time, the top-10 candidate passages are retrieved per query, then reranked by a BAAI/bge-reranker-base cross-encoder, keeping the top-3 (capped at 700 characters) as the final retrieved context.
- Retrieved context is prepended to the question and options in a structured prompt template before being passed to Qwen2.5-3B for answer scoring.

Model 3: RAG-Augmented QLoRA-Fine-Tuned Qwen2.5-3B-Instruct

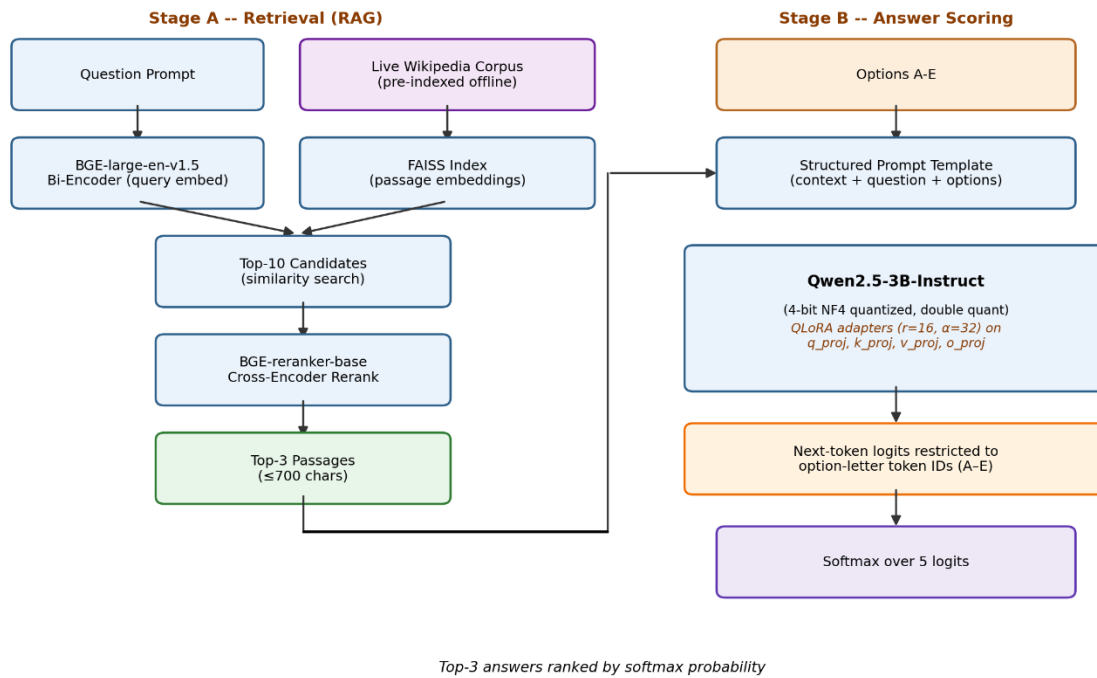


Figure 3. Architecture diagram of Model 3 - the two-stage pipeline: Stage A retrieves and reranks relevant Wikipedia passages (RAG); Stage B feeds the retrieved context, question, and options into QLoRA-fine-tuned Qwen2.5-3B for option-letter scoring.

Salient Points

- Retrieval augmentation directly addresses the dataset's lack of a shared context passage - rather than relying purely on parametric knowledge baked into the LLM's weights, the model is given task-relevant external context at inference time, similar in spirit to open-domain question answering systems.
- Rather than free-form text generation, inference restricts next-token prediction to the five option-letter token IDs (A-E) at the answer position, converting the generative model into an efficient, single-forward-pass, calibrated 5-way classifier - avoiding the cost and ambiguity of open-ended decoding.

Fine-Tuning Strategy (QLoRA)

- The base model was loaded in 4-bit NF4 precision with double quantization via bitsandbytes (QLoRA), dramatically reducing the GPU memory footprint needed to fine-tune a 3B-parameter model.
- LoRA adapters: rank $r=16$, $\alpha=32$, dropout=0.05, applied to the `q_proj`, `k_proj`, `v_proj`, and `o_proj` attention projection matrices.
- Training used loss masking: only the answer-letter token position contributed to the loss, so the model was trained specifically to predict the correct option letter rather than to reproduce the full prompt.
- Optimizer: AdamW with linear warmup (warmup_ratio=0.03); learning rate = $1e-4$; effective batch size 16 (per_device_train_batch_size=2, gradient_accumulation_steps=8); max_grad_norm=1 for gradient clipping.

- Mixed-precision (fp16) training with gradient checkpointing was used to fit training within available GPU memory (fp16 was used in preference to bf16, which was found to cause loss instability on the T4 GPUs used for training).
- Epochs: up to 5 (fewer than Models 1 and 2, reflecting the much larger model size and higher per-step cost), with early stopping (patience = 2) monitored on validation loss.

Quantization

4-bit quantization made it feasible to fine-tune a 3B-parameter model at all within the available hardware, at the cost of some precision - token-ID mappings for the option letters were explicitly verified before training, since tokenization of a leading space plus letter (e.g. “A”) is context- and tokenizer-dependent and must be confirmed rather than assumed.

6. Performance & Comparative Analysis

6.1 Evaluation Metrics

The primary metric, per the competition, is Mean Average Precision at 3 (MAP@3): for each question, a correct answer at predicted rank 1 scores 1.0, at rank 2 scores 0.5, at rank 3 scores 0.333, and 0 if outside the top-3, averaged across all questions. In addition to MAP@3, accuracy and macro-F1 were tracked during training for all models (logged to Weights & Biases per epoch) to allow common-metric comparison across the three architecturally different runs, as required by the course's experiment-tracking guidelines.

6.2 Training Performance

Each model's training and validation metrics (per-epoch train loss, validation accuracy, validation macro-F1, and validation MAP@3) were logged to Weights & Biases throughout training, under project dl-genai (entity: 23f3001593-dl-genai-project), with versioned model checkpoints saved as wandb. Artifact objects at each new best validation score.

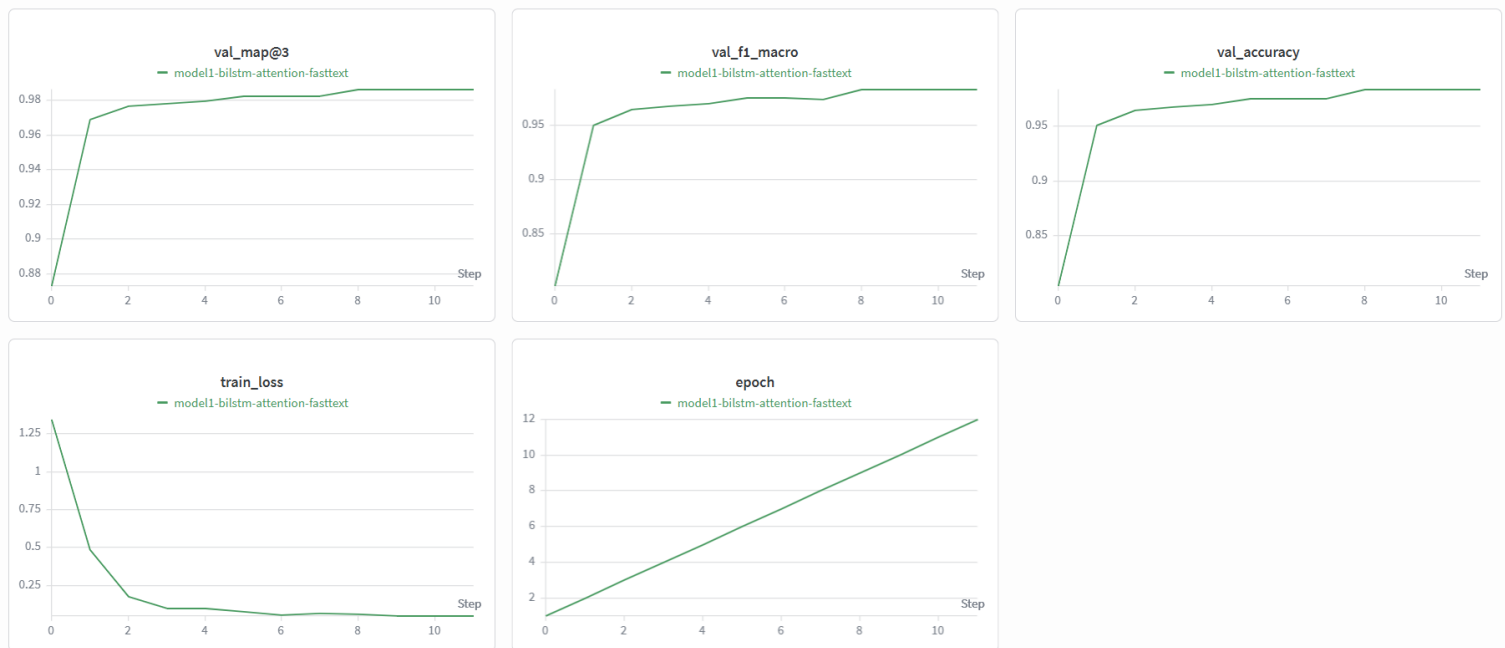


Figure 4. WandB visualizations of Model 1.



Figure 5. WandB visualizations of Model 2.



Figure 6. WandB visualizations of Model 3.

6.3 Comparative Report

The table below summarizes all three individually-trained models plus the final weighted ensemble. Submission MAP@3 (rather than validation MAP@3) is used for comparison, since validation scores were found to be unreliable for model-to-model and ensemble-weight comparisons - see Section 7 for discussion of this train/validation-versus-submission gap.

Model	Type	Trainable Params	Submission MAP@3	Ensemble Weight
Model 1: BiLSTM + Attention	From scratch	~1.2M (all)	0.7344	0.3229
Model 2: DeBERTa-v3-base + LoRA	Pretrained + PEFT	~0.9M (LoRA + heads) / 184M total	0.7535	0.3397
Model 3: Qwen2.5-3B + QLoRA + RAG	Generative LLM (model of choice)	~9.7M (LoRA) / 3.09B total (4-bit)	0.7510	0.3375
Weighted Ensemble	Combination of all three	—	0.7577 (Public LB)	—

Ensemble weighting method: weights were derived to be proportional to each model's “skill” above a random-guessing baseline (random MAP@3 for a 5-option, top-3 task = 0.367), computed directly from each model's Kaggle submission score rather than its validation score: $\text{weight}_i \propto \max(0, \text{submission_MAP3}_i - 0.367)$, normalized to sum to 1.

Trade-off discussion: Model 1 (from scratch) required no pretrained weights and trained fastest, but scored lowest, reflecting the ceiling of a lightweight architecture without transferred knowledge. Model 2 achieved the best individual score at the lowest computational and memory cost among the two transformer-based models (LoRA fine-tuning of a 184M-parameter encoder). Model 3 was by far the most computationally expensive to train and serve (3B parameters, 4-bit quantization, an additional retrieval pipeline), for a score comparable to, though slightly below, Model 2 - its value lies less in raw MAP@3 and more in architectural diversity for the ensemble, and in the generalizable retrieval-augmentation pattern it demonstrates. The near-equal final ensemble weights (0.32 / 0.34 / 0.34) indicate all three models ended up contributing comparably once evaluated on genuine held-out submission performance.

6.4 Kaggle Performance

Public leaderboard score: 0.75768 (MAP@3), Rank 320 / 1531 (top ~21%).

7. Conclusion & Future Work

7.1 Key Learnings

- Practical, hands-on experience with the full spectrum of modern fine-tuning approaches - full training from scratch, LoRA on a mid-sized encoder, and QLoRA (4-bit quantized) on a 3B-parameter generative model - clarified the real trade-offs between training cost, memory footprint, and downstream performance.
- Building a retrieval-augmented generation pipeline end-to-end (live corpus construction, embedding, indexing, reranking, prompt construction) demonstrated how external knowledge retrieval can substitute for a missing context passage in an MCQ task.
- Validation metrics can be unreliable indicators of true generalization, particularly for a complex, small-dataset fine-tuned generative model - this motivated using genuine held-out submission scores, rather than validation scores, to set ensemble weights.
- Deploying a multi-model pipeline in production surfaced real infrastructure constraints (GPU-only quantization libraries unavailable on CPU-only hosting, memory ceilings, cloud provider quota limits) that are easy to overlook when working purely within a training notebook.

7.2 Challenges Faced

- GPU memory constraints when fine-tuning a 3B-parameter model on limited hardware (Kaggle T4 GPUs) were addressed via QLoRA (4-bit quantization), gradient checkpointing, and small per-device batch sizes with gradient accumulation.
- bfloat16 mixed-precision training caused numerical instability on the available T4 GPUs (which lack full native bfloat16 support); switching to float16 resolved this.
- A significant gap between validation and submission scores for the individually strong-seeming Model 3 required moving away from validation-based ensemble weight optimization toward a submission-score-informed approach.
- Deployment surfaced a cascade of infrastructure issues: bitsandbytes 4-bit quantization requires a CUDA GPU and does not run on CPU-only hosting, requiring the QLoRA adapters to be merged into full-precision weights beforehand; container out-of-memory errors during model loading were resolved by switching Model 3 to float16 (rather than float32) on CPU and using `low_cpu_mem_usage=True` during model loading; and Google Cloud Run's regional memory quota (which reserves roughly double the configured memory during revision rollouts) required careful sizing of the deployed container's memory allocation.

7.3 Areas for Improvement

- Apply k-fold cross-validation, rather than a single train/validation split, to obtain a more robust estimate of each model's true generalization performance and reduce reliance on submission-score-informed ensembling.
- Explore stacking or learned rank-fusion ensembling (e.g. a small meta-model over the three models' per-option probabilities) rather than a fixed weighted average.
- Expand the RAG corpus beyond a single live Wikipedia snapshot - e.g. incorporating domain-specific reference sources relevant to the question topics observed in the dataset.
- Investigate quantization or distillation for Model 2 and Model 3 specifically to reduce inference latency and memory footprint in the deployed demo.
- Extend the deployed demo with batch-submission support and richer explainability (e.g. surfacing which retrieved passage most influenced Model 3's answer).

8. References

- Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735-1780.
- Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *ICLR 2015*.
- Bojanowski, P., Grave, E., Joulin, A., & Mikolov, T. (2017). Enriching Word Vectors with Subword Information. *Transactions of the ACL*, 5, 135-146. (FastText embeddings.)
- He, P., Liu, X., Gao, J., & Chen, W. (2020). DeBERTa: Decoding-enhanced BERT with Disentangled Attention. *arXiv:2006.03654*.
- He, P., Gao, J., & Chen, W. (2021). DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. *arXiv:2111.09543*.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. *arXiv:2106.09685*.
- Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. *arXiv:2305.14314*.
- Qwen Team, Alibaba Group (2024). Qwen2.5 Technical Report.
- Xiao, S., Liu, Z., Zhang, P., & Muennighoff, N. (2023). C-Pack: Packaged Resources To Advance General Chinese Embedding (BGE models). *arXiv:2309.07597*.

- Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs (FAISS). IEEE Transactions on Big Data.
- Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
- Hugging Face Transformers, PEFT, Datasets, and Sentence-Transformers library documentation. <https://huggingface.co/docs>
- Weights & Biases documentation. <https://docs.wandb.ai>
- Kaggle. Smart MCQ Solver Challenge competition page.